COMPLETE A-TO-Z CODE WALKTHROUGH & DIRECTORY REFERENCE

RestoPulse / Spot Light Biriyani Full Codebase Documentation

Complete engineering manual documenting every folder, every file, every TypeScript interface & helper method, database query execution & table schema, end-to-end request flow from `/login` through all operational POS screens, Super-Admin Dev Console, public pages, and Mobile / Tablet / Laptop responsive architecture.

8 + 19
8 Core Libraries (`src/lib/`) & 19 Route Files (`src/routes/`)

10 Tables
8 Prisma Models + 2 Live SaaS Config Tables in Neon PostgreSQL

50+ UI
Map, Brand Assistants, Layout Shell & 46 UI Primitives

COMPLETE TABLE OF CONTENTS

1. **System Architecture & Tech Stack** — Fullstack TanStack Start + Neon Cloud
2. **Complete Project Folder & File Directory** — Root, `public/`, `prisma/`, `src/*`
3. **Database Queries & 10-Table Inventory** — `neon-sync.ts` & `schema.prisma`
4. **End-to-End Request-Response & Query Lifecycles** — 6 Complete Data & SQL Flows
5. **`src/lib/` (All 8 Library Files)** — Store, Sync, Audit, Error Capture & Utils
6. **Server Entry, Auth Flow & Protected Shell** — `server.ts`, `start.ts`, `login.tsx`, `app.tsx`
7. **`src/components/` & `src/hooks/`** — Map, Chat Assistants, Shell & 46 UI Primitives
8. **Operational POS Routes (`/app/*`)** — Billing (8 Modals), Captain, Stock, Menu, Reports, Settings
9. **Super-Admin Dev Console & Public Routes** — `app.developer.tsx` + 7 Public Pages
10. **Mobile / Tablet / Laptop Matrix** — Exact Tailwind Responsive Breakpoints

1. System Architecture & Tech Stack

RestoPulse (operating **Spot Light Biriyani**, **Manam Restaurant**, and **Royal Brew Cafe & Bistro**) is a multi-tenant TypeScript SaaS POS and Restaurant Operating System built on **TanStack Start** + **React 19** + **Nitro** serverless functions, with offline-first browser persistence in `localStorage` and centralized PostgreSQL persistence on **Neon Cloud**.

Layer	Technology	Purpose in Project	Configured In
SSR & Server RPC	`@tanstack/react-start`, `nitro`	SSR rendering, file-based routes, and server-side `createServerFn` RPC bridge	`vite.config.ts`, `src/server.ts`, `src/start.ts`
Routing & UI	`@tanstack/react-router`, `react` 19.2	Generated typed route tree (`routeTree.gen.ts`), SPA transitions	`src/router.tsx`, `src/routes/*`
Design System	`tailwindcss` v4, `lucide-react`, `sonner`, `recharts`	Utility CSS breakpoints, icons, toast notifications, sales/profit charts	`src/styles.css`, `src/components/ui/*`
Client State	`localStorage`, `sessionStorage`, `BroadcastChannel`	Zero-latency POS billing, per-tab tenant isolation, real-time cross-tab sync	`src/lib/pos-store.ts`, `src/lib/audit-logger.ts`
Cloud Database	Neon Serverless PostgreSQL (AWS `ap-southeast-1`)	Shared multi-tenant SQL storage via Neon HTTPS `/sql` endpoint	`src/lib/neon-sync.ts`, `prisma/schema.prisma`



2. Complete Project Folder Structure & File Directory

Below is every folder and source file inside `spotlight-biriyani-full-kit/spotlight-biriyani/` and the exact responsibility of each file.

```
spotlight-biriyani/
├── .env / .env.example          # Environment config (DATABASE_URL, DATABASE_URL_POOLED, VITE_HOTLINE_PHONE)
├── package.json                 # Dependencies, scripts (dev, build, preview, start, lint)
├── vite.config.ts               # Vite + TanStack Start + Tailwind v4 + Nitro build pipeline
├── vercel.json                  # Vercel SPA rewrite rules (/(.*) → /index.html)
├── tsconfig.json                # TypeScript compiler options & @/* path alias → ./src/*
├── components.json              # shadcn/ui registry configuration
├── eslint.config.js / .prettierrc # Code formatting & lint rules
├── prisma/
│   └── schema.prisma            # PostgreSQL relational schema defining 8 core POS models
├── public/                      # Static root assets served directly at /
│   ├── favicon.ico, favicon.svg # App favicons
│   ├── manifest.json, sw.js     # PWA installability manifest & Service Worker
│   └── robots.txt, sitemap.xml  # Search engine crawler & sitemap configuration
├── scripts/
│   └── build-pdfs.mjs           # Node utility script to build documentation PDFs
└── src/
    ├── router.tsx               # TanStack Router instantiation (getRouter) + scroll restoration
    ├── routeTree.gen.ts         # Auto-generated typed route tree (never edited manually)
    ├── server.ts                # Custom Nitro/H3 server entry with error serialization wrapper
    ├── start.ts                 # Global TanStack Start errorMiddleware & csrfMiddleware hooks
    ├── styles.css               # Tailwind v4 imports, custom dark/light design tokens & print styles
    ├── assets/                  # 9 dish images (hero-biriyani.jpg, chicken-biriyani-real.jpg, mutton.jpg, etc.)
    ├── hooks/
    │   └── use-mobile.tsx       # Responsive viewport hook (<768px detection)
    ├── components/
    │   ├── map/ChennaiLeafletMap.tsx          # Interactive Chennai kitchen radar map (Spot Light & Manam pins)
    │   ├── site/SiteShell.tsx                 # Public marketing navbar, mobile drawer & footer
    │   ├── site/SpotlightAssistant.tsx        # Floating AI/preset helper widget for Spot Light customers
    │   ├── site/ManamAssistant.tsx            # Floating AI/preset helper widget for Manam late-night diners
    │   └── ui/*.tsx                           # 46 reusable UI primitives (Button, Dialog, Card, Table, Select, Chart, etc.)
    ├── lib/                     # Core Business Logic, Cloud SQL, State & Error Utilities (8 files)
    │   ├── pos-store.ts         # Multi-tenant DB store, types, profit/deg/forecast math, cross-tab bus
    │   ├── neon-sync.ts         # ServerFn PostgreSQL HTTP queries & bidirectional cloud sync
    │   ├── cloud-postgres.ts    # Cloud sync timestamp status metadata (slb.cloud.sync.meta)
    │   ├── audit-logger.ts      # 200-entry Dev Console & module-toggle audit history logger
    │   ├── error-capture.ts     # Captures & serializes full error stack/cause chains for server logs
    │   ├── error-page.ts        # Standalone fallback HTML 500 error page renderer
    │   ├── lovable-error-reporting.ts # Client React ErrorBoundary telemetry forwarder
    │   └── utils.ts             # ClassName helper cn() combining clsx + tailwind-merge
    └── routes/                  # File-based pages (19 routes)
        ├── __root.tsx           # Root HTML document shell, fonts, meta & Sonner Toaster
        ├── login.tsx            # 6-portal authentication gateway (Owner, Cashier, Waiter, Kitchen, Dev, Diner)
        ├── app.tsx              # Protected SaaS App shell, role/module nav guard, branch switcher, sync loop
        ├── app.index.tsx        # /app — Owner Executive Dashboard (KPIs & Recharts graphs)
        ├── app.billing.tsx      # /app/billing — Counter & Floor POS, KOT, UPI QR, Print, KDS, Z-Report
        ├── app.captain.tsx      # /app/captain — Waiter handheld tableside KOT ordering terminal
        ├── app.stock.tsx        # /app/stock — Raw material inventory & reorder threshold log
        ├── app.items.tsx        # /app/items — Per-tenant menu pricing, food cost & margin editor
        ├── app.reports.tsx      # /app/reports — Financial P&L, aggregator commission & 4-wk Chef Forecast
        ├── app.settings.tsx     # /app/settings — Store, 58/80mm printer, Swiggy/Zomato & JSON backup
        ├── app.developer.tsx    # /app/developer — Super-Admin tenant provisioning & branch entitlements
        ├── developer.tsx        # /developer — Guarded redirect entry into Developer portal
        └── index.tsx, menu.tsx, branches.tsx, contact.tsx, demo.tsx, platform.tsx, manam.tsx # Public pages
```

Root & Build Configuration Files Summary

File	What It Does
`package.json`	Declares dependencies (`react` , `@tanstack/react-start` , `recharts` , `lucide-react` , Radix UI packages) and scripts (`dev` , `build` , `start` , `lint`).
`vite.config.ts`	Configures Vite plugins: `devtools` , `vite-tsconfig-paths` , `@tailwindcss/vite` , `tanstackStart` , `nitro` , and React compiler/refresh.
`vercel.json`	Defines Vercel catch-all routing rewrite (`"/(.*)" → "/index.html"`) for client-side route resolution.
`.env` / `.env.example`	Stores `DATABASE_URL` , `DATABASE_URL_POOLED` , and `VITE_HOTLINE_PHONE` hotline variable.
`public/manifest.json` & `sw.js`	Enables PWA app installation and offline asset caching behavior on mobile/tablet POS browsers.
`src/assets/*`	9 restaurant product photos (Chicken/Mutton/Kushka biriyani, Bread Halwa, Kathirikai, Raita) imported into public pages.

3. Database Architecture: Where Queries Run & All 10 Tables

Exact Query Execution Flow

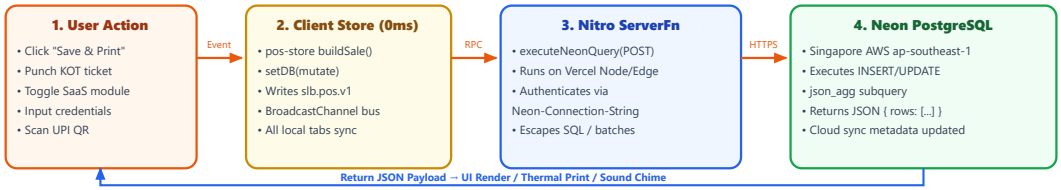
SQL queries do **not** execute inside the browser. All PostgreSQL queries live in `src/lib/neon-sync.ts` inside `executeNeonQuery = createServerFn({ method: "POST" })`. The React UI invokes helper functions in `neon-sync.ts`, which make an RPC POST call to the Nitro server. The server sends an HTTPS POST to `https://ep-small-frog-aorujp14.c-2.ap-southeast-1.aws.neon.tech/sql` authenticated with `Neon-Connection-String`. Note: `prisma/schema.prisma` documents the schema, while `@prisma/client` is not used at runtime.

3.1 Complete Inventory of All 10 Tables

#	Table Name	Defined In	Query Status	Columns & Role
1	`"Branch"`	`schema.prisma` + `neon-sync.ts`	ACTIVE NEON SQL	`id`, `name`, `address`, `phone`. Upserted before sales sync so foreign keys never fail.
2	`"DishItem"`	`schema.prisma` + `neon-sync.ts`	ACTIVE NEON SQL	`id`, `name`, `category`, `price`, `cost`, `active`, `createdAt`. Pulled & deduplicated in `performFullSync`.
3	`"Sale"`	`schema.prisma` + `neon-sync.ts`	ACTIVE NEON SQL	`id`, `date`, `branchId`, `channel`, `gross`, `discount`, `net`, `cogs`, `commission`, `packaging`, `profit`, `payment`, `customerPhone`, `customerName`, `notes`.
4	`"SaleLine"`	`schema.prisma` + `neon-sync.ts`	ACTIVE NEON SQL	`id`, `saleId` (FK), `itemId`, `name`, `qty`, `price`, `cost`. Pulled as nested JSON via `json_agg`.
5	`"TenantConfig"`	`neon-sync.ts` DDL	ACTIVE NEON SQL	`id`, `businessName`, `data JSONB` (stores modules, branches, pins, custom menu/stock), `updatedAt`.
6	`"PlatformConfig"`	`neon-sync.ts` DDL	ACTIVE NEON SQL	`key PK`, `value`, `updatedAt`. Stores global SaaS brand name (`saasPlatformName`).
7	`"Channel"`	`schema.prisma` only	SCHEMA / LOCAL	Channel payout commission rules (`counter`, `takeaway`, `swiggy`, `zomato`).
8	`"StockItem"`	`schema.prisma` only	SCHEMA / LOCAL	Raw ingredient stocks (name, unit, quantity, reorder threshold, costPerUnit).
9	`"StockMove"`	`schema.prisma` only	SCHEMA / LOCAL	Raw material purchase/kitchen-use/wastage ledger (`itemId`, `delta`, `type`, `note`).
10	`"KitchenBatch"`	`schema.prisma` only	SCHEMA / LOCAL	Lunch/Dinner capacity records (runtime computes plates left via `getLiveBatchStatus()`).

4. End-to-End Request-Response & Data Query Lifecycles

This section documents how a user action flows from the browser, through client state and server functions, into Neon PostgreSQL queries, and back as rendered UI data.



4.1 Detailed Data Lifecycle & Query Matrix

User Operation & Flow	Browser Input Data	Client Logic & Local Persistence	ServerFn & Exact SQL Query Executed	Response & UI Result
1. Staff Login & Role Authentication (`/login`)	`role: "cashier"|"owner"|"developer"` , `PIN / Password` , `tenantId` , `branchId` .	`handleLogin()` validates credentials against `currentTenant` in `db.tenants` . Writes `spotlight_session_active="true"` , `restopulse_session_tenant` , `spotlight_auth_role` , and updates active branch in `db.settings` .	Client-side authenticated. If role is developer, unlocks Super-Admin console. If staff, isolates tab context.	Toast `"Billing Terminal Unlocked"` + Redirects to `/app/billing?tenant=xxx` or `/app?tenant=xxx` .
2. Counter POS Bill Settlement (`/app/billing`)	`lines: [{ itemId, qty, price, cost }]` , `discount` , `payment: "cash"|"upi"|"card"` , `channel` , `customerPhone` , `tableId?` .	`buildSale()` calculates `gross` , `net` , `cogs` , `commission` , `profit` . `setDB()` appends to `db.sales` , decrements table occupancy, and broadcasts via `BroadcastChannel` .	`pushSalesToCloud([savedSale])` → `executeNeonQuery(POST)` : `INSERT INTO "Branch" ("id", "name", "address", "phone") VALUES (...) ON CONFLICT ("id") DO NOTHING;` `INSERT INTO "Sale" ("id", "date", "branchId", "channel", "gross", "discount", "net", "cogs", "commission", "packaging", "profit", "payment", "customerPhone", "notes") VALUES (...) ON CONFLICT ("id") DO UPDATE ...;` `INSERT INTO "SaleLine" ("id", "saleId", "itemId", "name", "qty", "price", "cost") VALUES (...) ON CONFLICT ("id") DO UPDATE ...;`	Plays Web Audio chime (`kaching`), triggers `printBill()` popup for 58mm/80mm thermal receipt, and decrements today's Deg batch.
3. Waiter Tableside KOT Punch (`/app/captain`)	`selectedTableId: "T1"` , `draftLines: [{ itemId, name, qty }]` , `orderNote` .	`handlePunchKot()` creates `kotId = kot_T1_timestamp` , calls `addKdsOrder()` with `status: "queued"` , updates table order via `setTableOrder()` , and triggers device haptic vibration (`25ms`).	Persisted locally in `db.tables` & `db.kdsOrders` . `BroadcastChannel` notifies Chef KDS tablet screen across the kitchen in real-time.	Chef screen displays new yellow KOT ticket. Waiter screen resets draft lines and marks table `Busy (🔴)` .
4. Super-Admin Module Entitlement Toggle (`/app/developer`)	`tenantId: "manam-restaurant"` , `branchId?` , `moduleKey: "billing"` , `enabled: false` .	`handleToggleModule()` updates `db.tenants[x].branches[y].enabledModules.billing = false` , logs entry to `logAuditEvent()` in circular buffer, and broadcasts state.	`pushTenantToCloud(updatedTenant)` → `executeNeonQuery(POST)` : `INSERT INTO "TenantConfig" ("id", "businessName", "data", "updatedAt") VALUES ('manam-restaurant', 'Manam Restaurant', '{...JSON...}', NOW()) ON CONFLICT ("id") DO UPDATE SET "data" = EXCLUDED."data", "updatedAt" = EXCLUDED."updatedAt";`	Manam Restaurant laptop running at Velachery automatically hides Billing screen on next sync / tab switch.
5. Live Biriyani Deg Batch Countdown	Daily cooked capacity (e.g. `200 plates`) from `db.settings.dailyBatchCapacity` + today's sales.	`getLiveBatchStatus()` filters `db.sales` where `isSameDay(s.date)` && `isDegItem(line)` . Calculates `soldToday` , `platesLeft = max(0, cap - soldToday)` , and splits into Chicken (`65%`) vs Mutton (`35%`).	Local calculation on synced sales. During 3:00 PM – 6:00 PM returns status `"Dinner Batch Ready at 7:00 PM"` .	Dish cards render live pill: `14 Plates Left` . If cart plates exceed remaining plates, order is blocked with warning banner.
6. 4-Week Weighted Chef Prep AI Forecast (`/app/reports`)	`itemId` , `targetDate = tomorrow` , historical `db.sales` .	`forecastQty()` extracts same-weekday sales over prior 4 weeks (weights: `[0.4, 0.3, 0.2, 0.1]`). Fallback: 14-day daily average × weekend surge factor (`1.2x` Fri–Sun, `1.05x` Mon–Thu).	Runs purely in memory on synced sales ledger without adding server load.	Reports table shows exact recommended cooking quantities (e.g., *Chicken Biriyani: 140 plates, Mutton: 85 plates*).

5. Complete Walkthrough of `src/lib/` (All 8 Files & Methods)

4.1 `src/lib/pos-store.ts` (1,999 lines) — Domain Types & Core Store

Type / Interface	Structure & Role
`Branch`, `Channel`, `Item`	Outlets (`id, name, address, phone`), sales channels (`commissionPct, packagingCost`), and menu dishes (`id, name, category, price, cost, active`).
`Sale` & `SaleLine`	Bill record with P&L breakdown (`gross, discount, net, cogs, commission, packaging, profit, payment, tenantId, branchId`) + line array.
`TableSession` & `KdsOrder`	Floor dining state (`id, status: vacant|occupied, section: main|ac|outdoor, capacity, lines[], occupiedSince`) and KDS kitchen tickets (`tokenNum, status: queued|cooking|ready, doneLineIndices[]`).
`TenantModules` & `TenantConfig`	13 module flags (`dashboard, billing, tables, captainPos, kds, stock, reports, menu, loyalty, dynamicUpi, zReport, multiBranch, aggregators`) + per-branch override map inside `TenantConfig` .
`ShiftDayClose` & `PrinterSettings`	Cashier Z-Report drawer closeout totals & thermal 58mm/80mm header/footer settings stored in root `DB` (`localStorage["slb.pos.v1"]`).

Exported Method	What It Does & How It Works
`getDB()` / `setDB(fn)` / `useDB()` / `resetDB()`	Reads/clones in-memory cache backed by `localStorage["slb.pos.v1"]` . `setDB` runs mutation, persists JSON, and notifies tabs via `BroadcastChannel("spotlight_pos_cross_tab_bus")` + custom DOM event `slb_store_updated` .
`exportJSON()` / `importJSON(raw)`	Exports formatted JSON backup string or parses/validates uploaded backup file into store.
`getActiveTenant(db?)`	Resolves active restaurant with strict tab isolation: (1) `?tenant=` query param → syncs to `sessionStorage["restopulse_session_tenant"]` ; (2) `sessionStorage` ; (3) fallback `db.activeTenantId` .
`getActiveBranchModules(tenant, brId)`	Merges default all- `true` modules → `tenant.enabledModules` → branch-specific `br.enabledModules` overrides.
`isDegBatchActiveForTenant(t, brId)`	Returns branch-level `degBatchEnabled` if defined, else tenant-level `t.degBatchEnabled !== false` .
`getTenantItems()` / `getTenantStock()`	Returns isolated menu & stock arrays per tenant; runs `cleanDishName()` & `sanitizeAndDeduplicateItems()` .
`switchClientTenant / updateTenant / addTenant / deleteTenant / toggleTenantModule / setSaasBrand`	CRUD & configuration mutators updating `db.tenants` , active session tenant, feature flags, and global white-label platform name.
`buildSale(input)`	Computes `gross` , `net` , `cogs` , channel `commission` , `packaging` , and net `profit = net − cogs − commission − packaging` .
`getLiveBatchStatus(db, brId, tId)`	Calculates remaining cooked biriyani capacity & Chicken/Mutton/Kushka split from today's 12h sales vs `dailyBatchCapacity` (200 default), detecting 3–6 PM prep window.
`filterSales / totals / itemBreakdown / dailySeries`	Reporting aggregators filtering by tenant/branch/date range and summarizing revenue, orders, avg ticket, and dish rankings.
`forecastQty(sales, itemId, date)`	4-week same-weekday weighted moving average (`[0.4, 0.3, 0.2, 0.1]`), falling back to 14-day daily average × weekend surge (`1.2x` Fri–Sun, `1.05x` Mon–Thu).
`getCustomerLoyalty(sales, phone)`	Aggregates total lifetime visits & spend for a 10-digit customer phone to unlock VIP rewards.
`setTableOrder / vacateTable / addTable / updateTable / deleteTable`	Manages Dine-in table lifecycle, running KOT items, section assignment, and table vacating.
`addKdsOrder / updateKdsStatus / toggleKdsLineDone / clearCompletedKdsOrders / clearAllKdsOrders / deleteKdsOrder / recordShiftClose`	Handles Kitchen Display order queue progression, individual dish strike-through, queue cleanup, and cashier Z-Report drawer close logging.

4.2 `src/lib/neon-sync.ts` (503 lines) — Cloud SQL Synchronization Engine

Method	Type & SQL Executed
`executeNeonQuery`	`createServerFn({ method: "POST" })` — Executes parameterized/raw SQL against Neon HTTP `/sql` endpoint using `NEON_CONN_STRING` .
`pullSalesFromCloud()`	`SELECT` on `"Sale"` + correlated `json_agg` subquery on `"SaleLine"` ordered by `"date" DESC LIMIT 500` .
`pushSalesToCloud(sales)`	Batches 50 sales at a time: upserts missing `"Branch"` rows (`ON CONFLICT DO NOTHING`), then upserts `"Sale"` and `"SaleLine"` rows.
`pullDishesFromCloud()`	`SELECT "id", "name", "category", "price", "cost", "active" FROM "DishItem" ORDER BY "name" ASC` + deduplication.
`pushTenantToCloud(t)` / `pushAllTenantsToCloud(list)` / `pushPlatformConfigToCloud(name)`	Ensures `"TenantConfig"` & `"PlatformConfig"` tables exist via `CREATE TABLE IF NOT EXISTS` and executes `INSERT ... ON CONFLICT DO UPDATE` for SaaS tenant JSON plans and platform branding.
`pullTenantsFromCloud()`	Reads all rows from `"TenantConfig"` & `"PlatformConfig"` , merges over `DEFAULT_TENANTS` , and backfills missing seed tenants.
`performFullSync()`	Locked orchestrator (`syncInProgress`) executing tenant pull → duplicate dish delete → dish pull → sales pull/merge → `setDB()` merge → `pushSalesToCloud` .

4.3 Remaining 6 Libraries in `src/lib/`

File	Exports & What Each Does
`cloud-postgres.ts`	`getCloudSyncStatus()` & `syncToCloudPostgres()` — Reads/updates `localStorage["slb.cloud.sync.meta"]` (last sync timestamp, endpoint label, synced row counts).

File	Exports & What Each Does
`audit-logger.ts`	`logAuditEvent(action, details, role?)`, `getAuditLogs()`, `clearAuditLogs()` — Stores a 200-entry circular audit log at `localStorage["restopulse_audit_logs_v1"]`.
`error-capture.ts`	`describeError()` & `consumeLastCapturedError()` — Wraps `console.error` and global error/unhandledrejection listeners to recover full stack + `cause` chains (up to depth 5 / 8,000 chars) swallowed by H3 500 responses.
`error-page.ts`	`renderErrorPage()` — Returns standalone clean HTML 500 error fallback document with Reload and Go Home buttons.
`lovable-error-reporting.ts`	`reportLovableError(error, context)` — Forwards React ErrorBoundary crashes (and Response status/URL failures) to preview telemetry hooks when available.
`utils.ts`	`cn(...inputs)` — Standard Tailwind class utility combining `clsx` conditional classes with `tailwind-merge` conflict resolution.

6. Server Entry Pipeline, Auth Flow & Protected Shell

Core Bootstrapping Files SRC/* ENTRY

- **`src/server.ts`:** Wraps `@tanstack/react-start/server-entry` with `normalizeError()` + H3 event normalization and invokes `consumeLastCapturedError()` on unhandled server throws.
- **`src/start.ts`:** Registers `errorMiddleware` and `csrfMiddleware` on `createStart()` for SSR & server-function protection.
- **`src/router.tsx`:** Exports `getRouter()` initializing `routeTree.gen.ts` with scroll restoration enabled.
- **`src/routes/__root.tsx`:** Renders root `<html>`, metadata, CSS (`styles.css`), Google Fonts, ErrorBoundary fallback, and Sonner `<Toaster />`.

6.1 Authentication Flow — `src/routes/login.tsx`

Portal / Role	Method & Validation	Storage Keys Set & Redirect Target
`owner`	`handleLogin` — Checks `currentTenant.loginPassword` or master codes (`9999` , brand passwords)	Sets `spotlight_session_active` , `restopulse_session_tenant` , `spotlight_auth_role="owner"` → `/app?tenant=<id>`
`cashier`	`handleLogin` — Validates `currentTenant.cashierPin` or `1234/1001/4001/0000` + selects branch	Updates `d.settings.activeBranchId` + `spotlight_auth_branch` → `/app/billing?tenant=<id>`
`waiter`	`handleLogin` — Validates Captain PIN (`2002` or assigned PIN)	Sets `spotlight_auth_role="waiter"` → `/app/captain?tenant=<id>`
`kitchen`	`handleLogin` — Validates `currentTenant.kitchenPin` or `5555`	Sets `spotlight_auth_role="kitchen"` → `/app/stock?tenant=<id>`
`developer`	`handleLogin` / `?portal=developer` — Validates Dev Key (`admin` , `9999` , `dev@spotlight2026`)	Sets `spotlight_auth_role="developer"` (tenant-independent) → `/app/developer`
`customer`	`handleCustomerSubmit()` — Captures mobile number & branch	Saves `spotlight_customer_phone` & redirects to `/manam` or `/menu`

6.2 Protected SaaS Shell — `src/routes/app.tsx`

- **`beforeLoad` Guard:** Checks active session token and tenant URL query param; redirects unauthorized access to `/login` .
- **Auto-Sync Effect:** Runs `performFullSync()` on mount, every `5 minutes` , on debounced `window` `focus` (at most once every 3 min), and on `online` events.
- **Dynamic Navigation (`visibleLinks`):** If `role === "developer"` , hides all restaurant tabs and renders **only** `Dev Console (/app/developer)` . For restaurant staff, filters `LINKS` by both role permissions and branch entitlements via `getActiveBranchModules(activeTenant, activeBranchId)` .

7. Components (`src/components/`) & Hooks (`src/hooks/`)

Folder & File	Component / Export	Responsibility
`src/hooks/use-mobile.tsx`	`useIsMobile()`	Tracks `window.matchMedia("(max-width: 767px)")` returning boolean for JS-driven mobile drawers/modals.
`src/components/map/ChennaiLeafletMap.tsx`	`ChennaiLeafletMap` , `CHENNAI_KITCHENS` , `MANAM_KITCHENS`	Interactive Chennai kitchen pin & radar map with branch switcher, live plate badges from `getLiveBatchStatus()` , and Google Maps links.
`src/components/site/SiteShell.tsx`	`SiteShell`	Shared public header navigation, mobile hamburger drawer, theme switcher, and company footer.
`src/components/site/SpotlightAssistant.tsx`	`SpotlightAssistant`	Floating Spot Light interactive Q&A chatbot (menu prices, batch times, party buckets, demo WhatsApp CTA).
`src/components/site/ManamAssistant.tsx`	`ManamAssistant`	Floating Manam Restaurant chatbot (late-night 4 AM dining info, Phoenix Market City directions, table WhatsApp booking, POS link).
`src/components/ui/*.tsx`	46 UI Primitives	Accessible Radix UI + Tailwind building blocks (`Button` , `Card` , `Dialog` , `Sheet` , `Drawer` , `Select` , `Tabs` , `Table` , `Chart` , `Sonner` , etc.).

8. Operational POS Routes (`/app/*`) — Page & Method Walkthrough

`app.index.tsx` (`/app`) — Executive Dashboard OWNER ANALYTICS

Guarded by `activeModules.dashboard` . Filters sales over `RANGES` (Today / 7d / 30d / All) using `filterSales()` & `totals()` to display 4 KPI cards, 2 Recharts revenue/profit & channel charts, and per-branch leaderboards.

`app.billing.tsx` (`/app/billing` · 4,106 lines) — Counter & Floor Billing Terminal — PRIMARY POS

Symbol / Method	What It Does & How It Works
`DishCard` & `Row`	Displays dish card with category icon, dynamic channel price, live remaining deg stock label, tap-to-add, and quantity stepper/input.
`add()` / `step()` / `updateQty()`	Checks `isDegItem()` and `liveBatch.platesLeft` capacity before incrementing biriyani/kushka items.
`addCustomItem()`	Creates off-menu / custom priced cart line on the fly via Custom Item modal.
`spotlight_active_table` sync	Reads active table handoff from Captain POS on mount, loads running KOT items, and enables Dine-in mode.
`saveOrUpdate()` & `handleKeyDown` (`F2`)	Calculates final P&L via `buildSale()`, saves to `setDB()`, pushes bill to Neon (`pushSalesToCloud`), auto-creates KDS ticket when enabled, vacates table if settling Dine-in, plays Web Audio `playChime()`, and calls `printBill()`. Bound to hotkey `F2`.
`startEditPastBill()` / `voidBill()`	Reloads any saved receipt into active cart for modification or deletes/voids a bill cleanly.
`printBill()` / `printKOT()` / `printZReport()`	Generates 58mm (210px) or 80mm (280–300px) formatted thermal HTML printouts for customer bills, dual-copy KOTs, and Z-Report shift closeouts.
8 Integrated Modals	`customModalOpen` (Open Item), `editBatchModalOpen` (Deg Capacity), `orderHistoryOpen` (Past Bills), `whatsappModalOpen` (WhatsApp Receipt), `tableFloorModalOpen` (Floor Plan & Add/Edit Table), `kdsModalOpen` (Kitchen Display KDS Board), `zReportModalOpen` (Cashier Day-Close), `upiQrModalOpen` (Dynamic exact-amount UPI QR via `api.qrserver.com`).

Route & File	Key Methods	What It Does & How It Works
`/app/captain` `app.captain.tsx`	`handlePunchKot()`, `handleGoToBilling()`, `handleVacateTable()`	Guarded by `activeModules.captainPos`. Waiter selects table → taps dishes → `handlePunchKot` sends ticket to KDS + saves running order onto table + plays chime/haptics; `handleGoToBilling` stores `spotlight_active_table` and hands off to cashier.
`/app/stock` `app.stock.tsx`	`applyMove()`, `addItem()`, `startEdit()`, `saveEdit()`, `deleteItem()`	Guarded by `activeModules.stock`. Manages raw ingredients per branch, highlights low-stock items (`qty ≤ reorderAt`), and logs Purchase/Out/Wastage moves into `db.moves`.
`/app/items` `app.items.tsx`	`update(id, patch)`, `removeItem(id)`, `add()`	Edits tenant-isolated menu prices, food costs, margin %, and active status with 6s undo delete toast; saves into tenant's `customItems` and pushes config.
`/app/reports` `app.reports.tsx`	`exportCsv()`, `exportPdf()`, `forecastQty()`	Calculates P&L after Swiggy/Zomato commissions & food cost; renders collapsible profit formulas box, tomorrow Chef Prep Forecast table, CSV download, and print PDF.
`/app/settings` `app.settings.tsx`	`update()`, `testPrint()`, `download()`, `upload()`	3 tabs (`store`, `printer`, `aggregators`): sets GST %, UPI VPA, branch selector, 58/80mm printer test print, aggregator fees, and JSON DB import/export.

9. Super-Admin Dev Console & Public Routes

- **`/app/developer` (`src/routes/app.developer.tsx`) & `/developer` (`src/routes/developer.tsx`):** Master-Key protected Super-Admin dashboard (`/developer` safely redirects to login/console). Resolves `activeScopedModules` for either *All Branches (Default Plan)* or individual outlets per tenant. Exposes `handleToggleModule()`, `handleToggleDegBatch()`, `handleApplyPresetPlan()` (Starter / Standard / Biriyani Pro / Enterprise), new tenant onboarding, credential inspection, `logAuditEvent()` audit viewer, and instant `pushTenantToCloud()` synchronization.
- **7 Public Routes:** `index.tsx` (Flagship homepage + live `BiriyaniCounter`), `menu.tsx` (Public INR price menu), `branches.tsx` (Chennai kitchens + interactive radar map), `contact.tsx` (Bulk catering contact cards), `demo.tsx` (Booking form → WhatsApp + ROI calculator), `platform.tsx` (RestoPulse SaaS showcase), and `manam.tsx` (Manam Restaurant showcase & menu).

10. Responsive Matrix: Mobile vs Tablet vs Laptop/Desktop

Screen / Route	Mobile (`<640px` / `<768px`)	Tablet (`sm: 640px` / `md: 768px`)	Laptop / Desktop (`lg: 1024px` / `xl: 1280px+`)
App Shell `app.tsx`	Compact mobile bar (`flex sm:hidden`) with abbreviated tenant title, branch icon, compact Sync/Theme/Logout buttons, and horizontal scrollable tab pills.	Desktop/Tablet bar reveals (`hidden sm:flex`) with full RestoPulse logo, branch switcher dropdown, Neon Cloud pill, and full sub-navbar.	Centered `max-w-7xl` workspace container with generous horizontal spacing.
Billing POS `app.billing.tsx`	Single-column stacked layout; horizontal scrollable categories; fixed bottom quick-checkout bar (`md:hidden fixed bottom-3`) with item count, net total, and jump-to-cart action.	At `md (768px)` activates 2-column grid `md:grid-cols-[minmax(0,1fr)_330px]`; vertical left category rail (`sm:w-36`) + 2-col dish cards (`sm:grid-cols-2`) + sticky cart (`md:sticky md:top-20`).	Cart widens to `lg: 360px` and `xl: 390px`; Top-Tab layout renders 3-col dish cards (`xl:grid-cols-3`); Floor Plan & KDS modals expand to 3–4 columns.
Captain POS `app.captain.tsx`	1-col dish cards (`grid-cols-1`); sticky bottom quick action bar (`lg:hidden`) to open KOT draft drawer on phone.	2-col dish cards (`sm:grid-cols-2`); expanded table title badges (`hidden sm:inline`) in header.	3-col dish cards (`xl:grid-cols-3`); mobile bottom bar hides and right sticky KOT ticket card stays visible.
Admin & Analytics	KPI cards, filter inputs, and tenant module tiles stack `grid-cols-1` cleanly.	2-column cards & module toggles (`sm:grid-cols-2` / `md:grid-cols-2`); 4-col report filters (`sm:grid-cols-4`).	4-col Dashboard KPIs (`lg:grid-cols-4`), side-by-side charts, stock `[1fr_380px]` split, and 3-col tenant grid (`xl:grid-cols-3`).